Chapitre 4 : Création de Graphiques de Base avec D3.js

Par Robert DIASSÉ

4.1 Gestion des Données avec D3.js

Avant de créer des graphiques, il est essentiel de comprendre comment D3.js gère les données. Les graphiques sont basés sur des données liées aux éléments DOM, et D3 fournit des méthodes puissantes pour manipuler ces données.

4.1.1 Liaison des Données

La méthode `data()` est utilisée pour lier un tableau de données aux éléments DOM. Cette liaison est essentielle pour créer des graphiques dynamiques.
Exemple :

```
const data = [10, 20, 30, 40]; // Données à lier
d3.selectAll("rect")           // Sélection des rectangles
  .data(data)                 // Liaison des données
  .attr("width", d => d * 10) // Modifier l'attribut 'width' en fonction des
données
  .attr("height", 20);
```

4.1.2 Méthodes Clés

- `enter()` : Crée de nouveaux éléments pour les données supplémentaires.
- `exit()` : Supprime les éléments lorsque les données sont réduites.
- `merge()` : Combine les sélections existantes et les nouvelles.

4.2 Les Échelles dans D3.js

Les échelles sont des fonctions essentielles qui permettent de mapper les valeurs des données à des dimensions visuelles (comme la position, la couleur, ou la taille). Cela garantit que les graphiques sont bien proportionnés et adaptés aux données.

4.2.1 Types d'Échelles dans D3.js

D3.js propose différents types d'échelles, chacune adaptée à un type de données spécifique.

Type d'échelle	Description	Cas d'utilisation
Échelle linéaire (`d3.scaleLinear`)	Mappe une plage de valeurs continues proportionnellement.	Graphiques linéaires, barres, bulles.
Échelle logarithmique (`d3.scaleLog`)	Transforme les données selon une échelle logarithmique (utile pour les grands écarts).	Données exponentielles (population, richesse).
Échelle de catégorie (`d3.scaleBand`)	Divise les données en catégories discrètes.	Graphiques en barres ou diagrammes à bandes.
Échelle de couleur (`d3.scaleOrdinal`)	Mappe des catégories à des couleurs spécifiques.	Cartes thématiques, catégories nominales.

4.2.2 Création d'une Échelle Linéaire

Voici un exemple de création d'une échelle linéaire pour une donnée allant de 0 à 100 et mappée à une plage de 0 à 500 pixels :

```
const scale = d3.scaleLinear()
  .domain([0, 100])   // Domaine des données d'entrée
  .range([0, 500]);   // Plage des valeurs de sortie
```

- `domain([min, max])` : Les valeurs minimales et maximales des données.
- `range([min, max])` : Les dimensions de sortie, souvent en pixels.

Utilisation :

```
console.log(scale(50));  // Retourne 250
console.log(scale(100)); // Retourne 500
```

4.2.3 Création d'une Échelle de Catégorie

Pour les données catégoriques, nous utilisons `d3.scaleBand()`.
Exemple :

```
const categories = ["A", "B", "C", "D"];
const scaleBand = d3.scaleBand()
  .domain(categories)       // Catégories d'entrée
```

```
    .range([0, 400])          // Plage de sortie (en pixels)
    .padding(0.1);            // Ajoute de l'espace entre les bandes
```

Utilisation :

```
console.log(scaleBand("A")); // Retourne la position de la catégorie "A"
console.log(scaleBand.bandwidth()); // Retourne la largeur de chaque bande
```

4.3 Gestion des Axes dans D3.js

Les axes sont des éléments graphiques essentiels qui aident à rendre un graphique lisible. Ils sont basés sur les échelles définies précédemment.

4.3.1 Création d'Axes

D3.js offre quatre générateurs d'axes :

- `d3.axisTop()` : Place les ticks en haut du graphique.
- `d3.axisBottom()` : Place les ticks en bas (souvent utilisé pour l'axe des x).
- `d3.axisLeft()` : Place les ticks à gauche (souvent utilisé pour l'axe des y).
- `d3.axisRight()` : Place les ticks à droite.

4.3.2 Les Ticks : Personnalisation et Gestion Avancée

Les ticks (graduation) sont des marques ou étiquettes placées le long des axes pour aider les utilisateurs à interpréter les données. Dans D3.js, il est possible de personnaliser le nombre, le format et l'emplacement des ticks.

Options des Ticks :

1. **Nombre de ticks (`ticks(n)`)** : Permet de définir un nombre approximatif de divisions.

   ```
   xAxis.ticks(10); // Approximation à 10 ticks
   ```

2. **Formateur des ticks (`tickFormat`)** : Permet de changer l'apparence des ticks.
   Exemple : Ajouter un symbole de pourcentage :

   ```
   xAxis.tickFormat(d => `${d}%`);
   ```

3. **Valeurs personnalisées (`tickValues`)** : Affiche des valeurs spécifiques uniquement.

   ```
   xAxis.tickValues([0, 25, 50, 75, 100]);
   ```

4. **Placement des ticks :**

- Ajustez leur position à l'aide de `tickSizeInner` et `tickSizeOuter`.
  Exemple :

```
xAxis.tickSizeInner(-300).tickSizeOuter(5);
```

4.4 Mise en Place de Graphiques de Base

En combinant les notions de données, échelles et axes, nous pouvons créer différents types de graphiques.

4.4.1 Graphique en Barres

Un graphique en barres représente des données catégoriques à l'aide de rectangles.

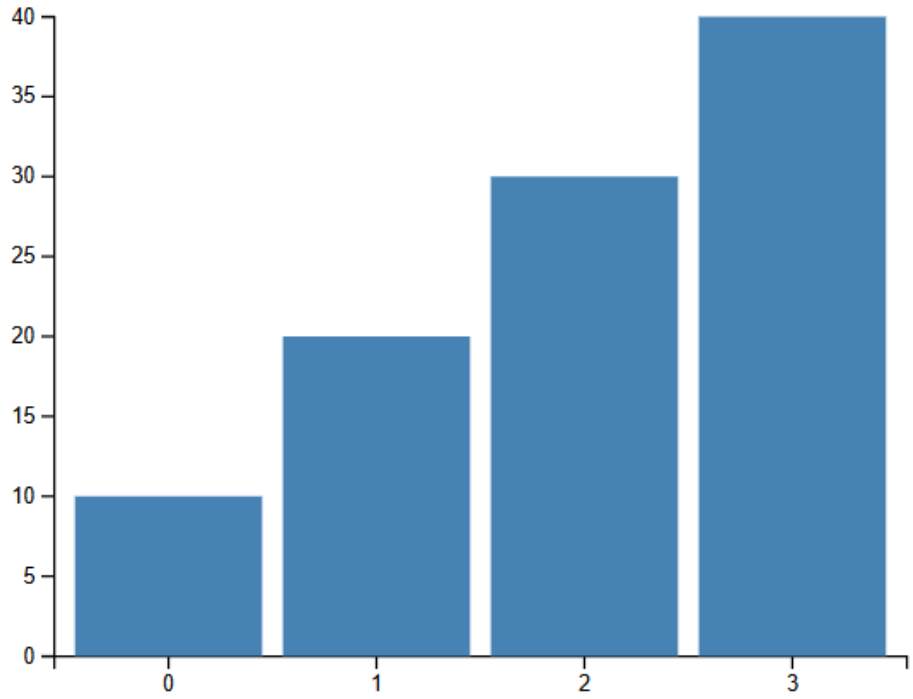
Étapes :

1. Créer une échelle de catégorie pour l'axe x.
2. Créer une échelle linéaire pour l'axe y.
3. Générer les axes.
4. Ajouter les barres en fonction des données.

Exemple :

```
const data = [10, 20, 30, 40];

const xScale = d3.scaleBand()
  .domain(data.map((d, i) => i)) // Catégories basées sur l'index
  .range([0, 400])
  .padding(0.1);

const yScale = d3.scaleLinear()
  .domain([0, d3.max(data)])
  .range([300, 0]);

const xAxis = d3.axisBottom(xScale);
const yAxis = d3.axisLeft(yScale);

svg.append("g")
  .attr("transform", "translate(0, 300)")
  .call(xAxis);

svg.append("g").call(yAxis);

svg.selectAll("rect")
  .data(data)
  .enter()
```

```
  .append("rect")
  .attr("x", (d, i) => xScale(i))
  .attr("y", d => yScale(d))
  .attr("width", xScale.bandwidth())
  .attr("height", d => 300 - yScale(d))
  .attr("fill", "steelblue");
```



4.4.2 Graphique Linéaire (Courbes)

Un graphique linéaire représente une série de points connectés par des lignes.

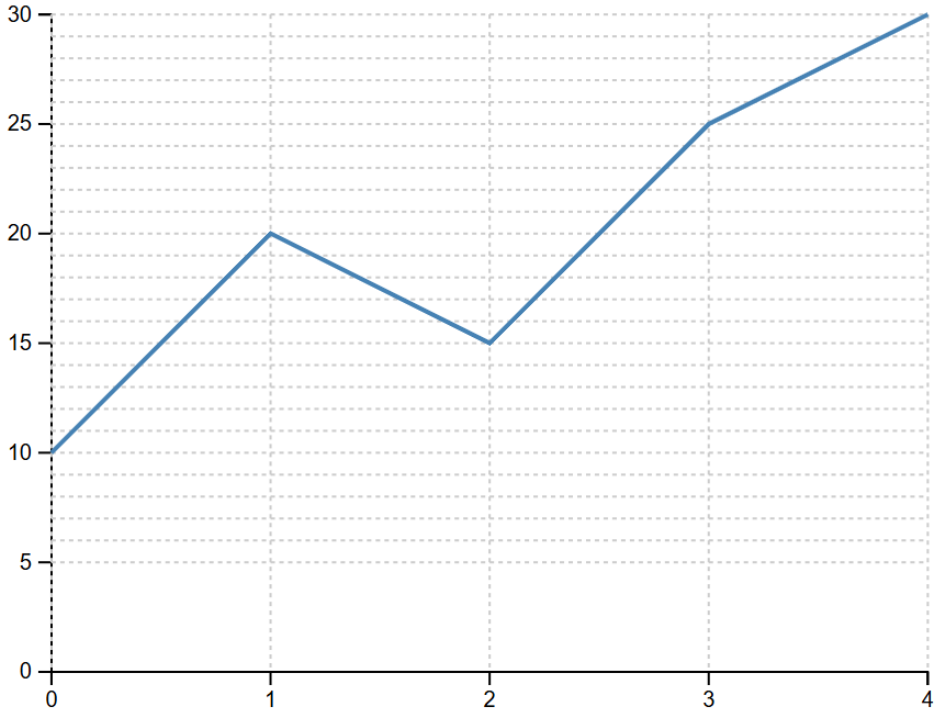
Étapes :

1. Utiliser une échelle linéaire pour les axes x et y.
2. Ajouter les axes au graphique.
3. Tracer une ligne connectant les points.

Exemple :

```
const data = [10, 20, 15, 25, 30];

const xScale = d3.scaleLinear()
  .domain([0, data.length - 1])
  .range([0, 400]);

const yScale = d3.scaleLinear()
  .domain([0, d3.max(data)])
  .range([300, 0]);
```

```
const line = d3.line()
  .x((d, i) => xScale(i))
  .y(d => yScale(d));

svg.append("path")
  .datum(data)
  .attr("d", line)
  .attr("fill", "none")
  .attr("stroke", "steelblue")
  .attr("stroke-width", 2);
```



4.4.3 Graphique Circulaire

Un graphique circulaire, ou camembert, représente des proportions d'un tout à l'aide de secteurs.

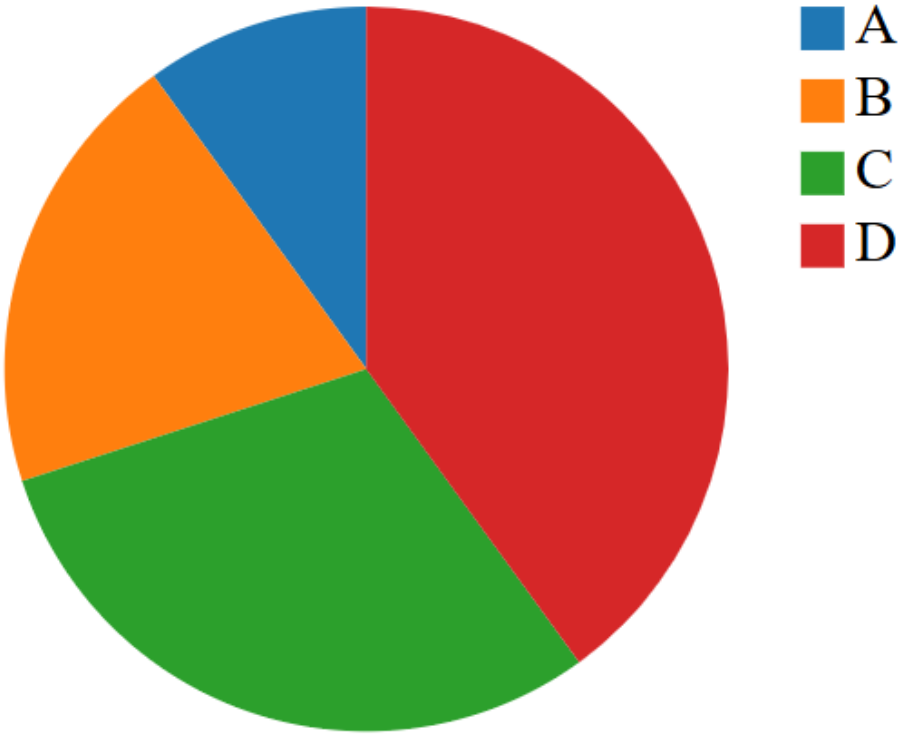
Étapes :

1. Créer une fonction d'arc pour définir les secteurs.
2. Calculer les angles des secteurs en fonction des données.
3. Ajouter les arcs au graphique.

Exemple :

```
const data = [10, 20, 30, 40];
const color = d3.scaleOrdinal(d3.schemeCategory10); // Palette de couleurs

const pie = d3.pie();
const arc = d3.arc()
  .innerRadius(0) // Pour un camembert, le rayon intérieur est 0
```

```
    .outerRadius(100); // Rayon extérieur de 100 pixels

  const g = svg.append("g")
    .attr("transform", "translate(200, 150)"); // Centrer le camembert

  g.selectAll("path")
    .data(pie(data))
    .enter().append("path")
    .attr("d", arc)
    .attr("fill", (d, i) => color(i));
```



4.4.4 Nuage de Points

Un nuage de points (scatter plot) affiche des points sur un plan pour représenter deux variables numériques.

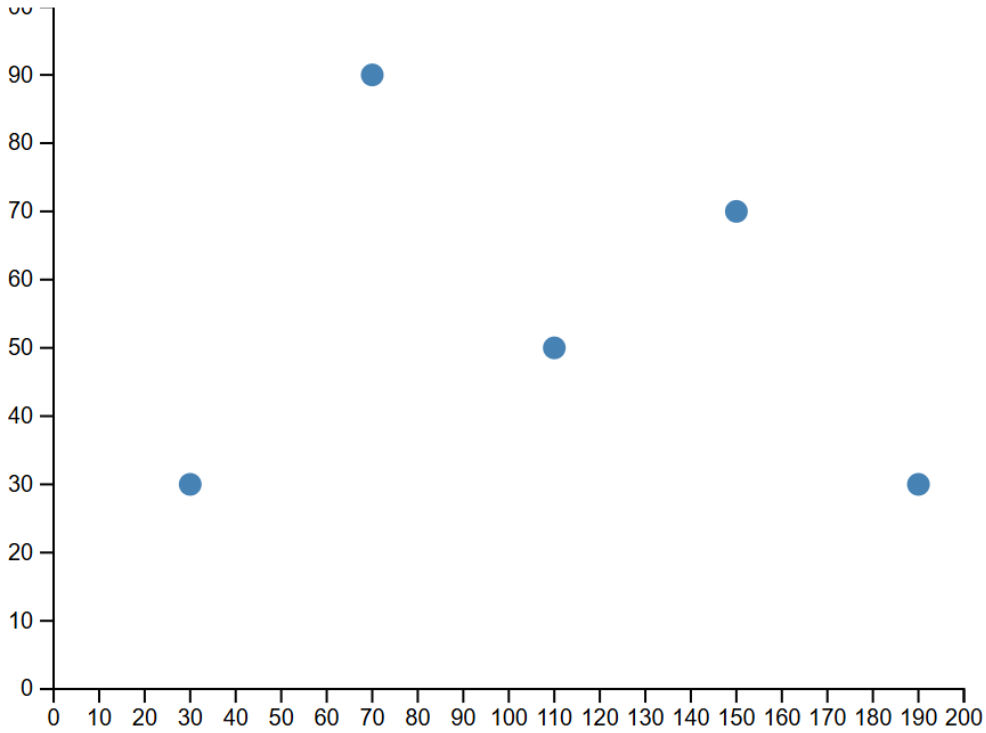
Étapes :

1. Créer des échelles linéaires pour les axes x et y.
2. Ajouter les axes au graphique.
3. Placer les points en fonction des données.

Exemple :

```
  const data = [
    { x: 30, y: 30 },
```

```
    { x: 70, y: 90 },
    { x: 110, y: 50 },
    { x: 150, y: 70 },
    { x: 190, y: 30 }
  ];

  const xScale = d3.scaleLinear()
    .domain([0, 200])
    .range([0, 400]);

  const yScale = d3.scaleLinear()
    .domain([0, 100])
    .range([300, 0]);

  svg.append("g")
    .attr("transform", "translate(0, 300)")
    .call(d3.axisBottom(xScale));

  svg.append("g")
    .call(d3.axisLeft(yScale));

  svg.selectAll("circle")
    .data(data)
    .enter().append("circle")
    .attr("cx", d => xScale(d.x))
    .attr("cy", d => yScale(d.y))
    .attr("r", 5)
    .attr("fill", "steelblue");
```



4.4.5 Cas d'Utilisation pour Chaque Graphique

Type de Graphique	Cas d'utilisation
Graphique en barres	Comparaison de valeurs discrètes (catégories, fréquences).
Graphique linéaire	Suivi de tendances sur une période ou une série ordonnée.
Graphique circulaire	Répartition proportionnelle d'un total (camembert, donut).

4.5 Conclusion

Ce chapitre a présenté les concepts fondamentaux pour la création de graphiques de base avec D3.js, en mettant un accent particulier sur les échelles et les axes. En comprenant ces éléments, il devient possible de visualiser des données complexes de manière efficace et esthétique.